

Ferramentas Utilizadas no TCC: Algoritmos de Compressão de Dados

Gustavo Yujii Silva Kadooka

Universidade Estadual Paulista - Campus Bauru

2025

- 1 Visão Geral do Projeto
- 2 Ferramentas de Desenvolvimento
- 3 C++ e Estruturas de Dados
- 4 Algoritmo de Huffman - Estruturas de Dados
- 5 GTK para Interface Gráfica
- 6 Integração das Ferramentas
- 7 Implementação Real do Projeto GKompress
- 8 Conclusões

Tema

Análise comparativa de algoritmos de compressão de dados clássicos

Algoritmos Implementados

- **Huffman** (foco principal)
- LZ77
- LZW
- GZIP

Objetivo

Comparar eficiência em taxa de compressão e tempo de execução

Back-End

- **C++** - Linguagem principal
- **STL** - Estruturas de dados
- **Make** - Build system
- **Git** - Controle de versão

Front-End

- **GTK** - Interface gráfica

Análise

- **Python** - Geração de gráficos
- **Matplotlib** - Visualização

Por que C++?

Vantagens para o projeto

- **Performance:** Controle total sobre memória e processamento
- **STL:** Estruturas de dados otimizadas (`priority_queue`, `map`, `vector`)
- **Portabilidade:** Código executável em diferentes plataformas
- **Compatibilidade:** Integração com bibliotecas C existentes

Desafios

- Gerenciamento manual de memória
- Complexidade de sintaxe
- Debugging mais complexo

Estruturas de Dados Fundamentais em C++

```
1 #include <vector>           // Arrays dinamicos
2 #include <map>              // Dicionarios/Mapas
3 #include <queue>            // Filas
4 #include <priority_queue>   // Filas de prioridade
5 #include <string>           // Strings
6 #include <fstream>          // Manipulacao de arquivos
7
8 // Exemplo de uso basico
9 std::vector<int> frequencias(256, 0);
10 std::map<char, std::string> codigos;
11 std::priority_queue<Node*, std::vector<Node*>, Compare> heap;
12
```

Listing 1: Principais estruturas utilizadas

Visão Geral do Algoritmo de Huffman

Princípio

Codificação baseada na frequência de ocorrência dos símbolos

Etapas

- 1 Análise de frequência dos símbolos
- 2 Construção da árvore de Huffman
- 3 Geração dos códigos
- 4 Codificação/Decodificação

Estrutura do Nó da Árvore de Huffman

```
1 struct Node {
2     char symbol;           // Símbolo (para folhas)
3     int frequency;         // Frequência do símbolo
4     Node* left;            // Filho esquerdo
5     Node* right;           // Filho direito
6
7     // Construtor para folhas
8     Node(char s, int f) : symbol(s), frequency(f),
9                          left(nullptr), right(nullptr) {}
10
11    // Construtor para nós internos
12    Node(int f, Node* l, Node* r) : symbol('\0'), frequency(f),
13                                   left(l), right(r) {}
14
15    // Destrutor
16    ~Node() {
17        delete left;
18        delete right;
19    }
20 };
21
```

Listing 2: Definição da estrutura Node

Heap de Prioridade para Construção da Árvore

```
1 struct Compare {
2     bool operator()(Node* a, Node* b) {
3         return a->frequency > b->frequency; // Min-heap
4     }
5 };
6
7 // Declaracao da fila de prioridade
8 std::priority_queue<Node*, std::vector<Node*>, Compare> minHeap;
9
10 // Uso na construcao da arvore
11 while (minHeap.size() > 1) {
12     Node* left = minHeap.top(); minHeap.pop();
13     Node* right = minHeap.top(); minHeap.pop();
14
15     Node* merged = new Node(left->frequency + right->frequency,
16                             left, right);
17     minHeap.push(merged);
18 }
19
```

Listing 3: Comparador para priority_queue

Mapeamento de Códigos

```
1 class HuffmanEncoder {
2 private:
3     std::map<char, std::string> huffmanCodes;
4     std::map<std::string, char> reverseCodes; // Para decodificacao
5
6 public:
7     // Geracao dos codigos atraves de travessia da arvore
8     void generateCodes(Node* root, std::string code = "") {
9         if (!root) return;
10
11         if (!root->left && !root->right) { // Folha
12             huffmanCodes[root->symbol] = code;
13             reverseCodes[code] = root->symbol;
14             return;
15         }
16
17         generateCodes(root->left, code + "0");
18         generateCodes(root->right, code + "1");
19     }
20
21     // Codificacao de um texto
22     std::string encode(const std::string& text) {
23         std::string result = "";
24         for (char c : text) {
25             result += huffmanCodes[c];
26         }
27         return result;
28     }
29 };
30
```

Listing 4: Armazenamento dos códigos de Huffman

Array de Frequências

- `std::vector<int> frequencias(256, 0)`
- Acesso direto por índice (ASCII)
- Complexidade: $O(1)$ para acesso

Mapa de Frequências (alternativa)

- `std::map<char, int> freqMap`
- Economia de memória para textos com poucos símbolos únicos
- Complexidade: $O(\log n)$ para acesso

Implementação da Análise de Frequência

```
1 void analyzeFrequency(const std::string& filename,
2                       std::vector<int>& frequencias) {
3     std::ifstream file(filename, std::ios::binary);
4     if (!file.is_open()) {
5         throw std::runtime_error("Erro ao abrir arquivo");
6     }
7
8     char byte;
9     while (file.get(byte)) {
10         frequencias[static_cast<unsigned char>(byte)]++;
11     }
12
13     file.close();
14 }
15
16 // Uso
17 std::vector<int> frequencias(256, 0);
18 analyzeFrequency("input.txt", frequencias);
19
```

Listing 5: Contagem de frequências

Buffer de Bits

- Necessário para armazenar códigos de tamanho variável
- Implementação com `std::vector<bool>` ou manipulação bit-a-bit

Metadados do Arquivo

- Tabela de frequências (para reconstruir a árvore)
- Tamanho original do arquivo
- Número de bits válidos no último byte

Buffer de Bits - Escrita

```
1 class BitBuffer {
2 private:
3     std::vector<uint8_t> data;
4     int bitPosition;
5
6 public:
7     void writeBit(bool bit) {
8         if (bitPosition % 8 == 0) {
9             data.push_back(0);
10        }
11
12        if (bit) {
13            data.back() |= (1 << (7 - (bitPosition % 8)));
14        }
15        bitPosition++;
16    }
17
18    void writeBits(const std::string& bits) {
19        for (char bit : bits) {
20            writeBit(bit == '1');
21        }
22    }
23 };
24
```

Listing 6: Classe para escrita de bits

```
1  bool readBit() {
2      int byteIndex = bitPosition / 8;
3      int bitIndex = bitPosition % 8;
4      bitPosition++;
5
6      return (data[byteIndex] >> (7 - bitIndex)) & 1;
7  }
8
9  void flush() {
10     // Flush remaining bits
11     if (bitPosition % 8 != 0) {
12         bitPosition = (bitPosition / 8 + 1) * 8;
13     }
14 }
15 };
16
```

Listing 7: Métodos de leitura de bits

Por que GTK?

Vantagens

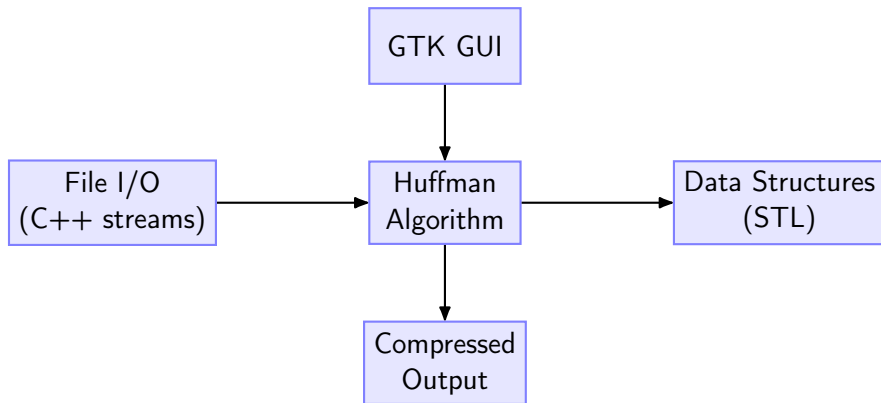
- **Cross-platform:** Linux, Windows, macOS
- **Nativo:** Integração com sistema operacional
- **Performance:** Interface responsiva
- **Documentação:** Bem documentado e maduro

Estrutura do Front-End

- `GUI.cpp` - Lógica da interface
- `GUI.hpp` - Definições das classes
- Glade - Designer visual (opcional)


```
1 class HuffmanGUI {
2 private:
3     GtkWidget* window;
4     GtkWidget* fileChooser;
5     GtkWidget* compressButton;
6     GtkWidget* decompressButton;
7
8 public:
9     HuffmanGUI() {
10         gtk_init(NULL, NULL);
11         createWindow();
12         setupWidgets();
13         connectSignals();
14     }
15
16     void createWindow() {
17         window = gtk_window_new(GTK_WINDOW_TOPLEVEL);
18         gtk_window_set_title(GTK_WINDOW(window),
19                             "Huffman Compressor");
20         gtk_window_set_default_size(GTK_WINDOW(window), 600, 400);
21     }
22 };
23
```

Listing 8: Classe principal da GUI



- 1 Interface **GTK** recebe seleção de arquivo
- 2 **C++ File I/O** lê o arquivo original
- 3 **Análise de Frequência** usando `std::vector<int>`
- 4 **Construção da Árvore** usando `std::priority_queue`
- 5 **Geração de Códigos** usando `std::map`
- 6 **Codificação** com `BitBuffer` customizado
- 7 **Salvamento** do arquivo comprimido

Estrutura do Código

- **Herança:** Classe base Huffman com especializações
- **Polimorfismo:** HuffmanTXT, HuffmanBMP, HuffmanWAV
- **Templates:** Uso de `std::array`, `std::vector`, `std::map`
- **RAII:** Gerenciamento automático de recursos

Padrões de Design

- **Strategy Pattern:** Diferentes algoritmos de compressão
- **Template Method:** Fluxo comum de compressão/descompressão
- **Factory Pattern:** Criação de instâncias por tipo de arquivo

Estruturas de Dados Personalizadas - Huffman

```
1 struct Node {
2     uint8_t byte;           // Simbolo (para folhas)
3     uint32_t duplicates;    // Frequencia do simbolo
4     Node* left;             // Filho esquerdo
5     Node* right;            // Filho direito
6     Node* next;             // Para lista ligada (min-heap manual)
7
8     Node(uint8_t b, uint32_t dup) :
9         byte(b), duplicates(dup),
10         left(nullptr), right(nullptr), next(nullptr) {}
11
12     int is_leaf() {
13         return (left == nullptr && right == nullptr) ? 1 : 0;
14     }
15 };
16
```

Listing 9: Estrutura Node personalizada do projeto

Lista Ligada Ordenada - Min-Heap Manual

```
1 class List {
2 public:
3     Node* front = nullptr;
4     uint32_t size = 0;
5
6     // Insere no mantendo ordem crescente por frequencia
7     void insert_sorted(Node* node) {
8         if(front == nullptr) {
9             front = node;
10        } else if(node->duplicates < front->duplicates) {
11            node->next = front;
12            front = node;
13        } else {
14            Node* aux = front;
15            while(aux->next && aux->next->duplicates < node->duplicates)
16                aux = aux->next;
17            node->next = aux->next;
18            aux->next = node;
19        }
20        size++;
21    }
22
23    Node* remove_front() {
24        Node* remove = nullptr;
25        if(front != nullptr) {
26            remove = front;
27            front = remove->next;
28            remove->next = nullptr;
29            size--;
30        }
31        return remove;
32    }
33 }
```

Construção da Árvore de Huffman

```
1 void HuffmanTree::build(List& list) {
2     Node *first_removed, *second_removed, *new_node;
3
4     // Repete ate restar apenas um no (raiz)
5     while(list.size > 1) {
6         // Remove os dois nos com menor frequencia
7         first_removed = list.remove_front();
8         second_removed = list.remove_front();
9
10        // Cria novo no interno
11        new_node = new Node(' ',
12            first_removed->duplicates + second_removed->duplicates);
13        new_node->left = first_removed;
14        new_node->right = second_removed;
15        new_node->next = nullptr;
16
17        // Reinsere na lista ordenada
18        list.insert_sorted(new_node);
19    }
20    root = list.front();
21 }
22
```

Listing 11: Algoritmo de construção da árvore

Compressão BMP - Parte 1

```
1 void HuffmanBMP::compress(int option) {
2     // Le arquivo BMP com headers
3     bmp_file.read(filepath.c_str());
4
5     // Conta frequencias apenas dos dados de imagem
6     count_duplicates(bmp_file.data, bmp_file.data_size);
7
8     // Constroi arvore e dicionario
9     list.fill(duplicates);
10    tree.build(list);
11    size_t max_columns = tree.height();
12    dictionary.build(max_columns);
13    dictionary.fill(tree.root, "", max_columns, option);
14
15    // Codifica apenas os dados, preserva headers
16    char* code = encode(bmp_file.data, bmp_file.data_size);
17    internal_compress(option, code, write_bmp_header, &bmp_file);
18 }
19
```

Listing 12: Leitura e análise de frequências

Compressão BMP - Parte 2

```
1 // Funcao callback para escrever headers BMP
2 static void write_bmp_header(FILE* file, void* image_file) {
3     bmp* bmp_ptr = static_cast<bmp*>(image_file);
4     fwrite(&bmp_ptr->header, sizeof(bmp_ptr->header), 1, file);
5     fwrite(&bmp_ptr->info_header, sizeof(bmp_ptr->info_header), 1, file);
6     if(bmp_ptr->has_color_table()) {
7         fwrite(bmp_ptr->color_table,
8             bmp_ptr->get_color_table_size(), 1, file);
9     }
10 }
11
```

Listing 13: Codificação e salvamento

Complexidade Temporal

- Análise de frequência: $O(n)$
- Construção da árvore: $O(k \log k)$ onde k = símbolos únicos
- Codificação: $O(n)$
- **Total: $O(n + k \log k)$**

Complexidade Espacial

- Array de frequências: $O(1)$ - tamanho fixo
- Árvore de Huffman: $O(k)$
- Mapa de códigos: $O(k)$
- **Total: $O(k)$**

STL - Standard Template Library

- **priority_queue**: Heap otimizado para construção da árvore
- **map**: Busca eficiente de códigos $O(\log n)$
- **vector**: Arrays dinâmicos com acesso $O(1)$

C++ vs Outras Linguagens

- **Performance**: Controle total sobre memória
- **Portabilidade**: Código executável nativo
- **Integração**: Compatibilidade com GTK

Estruturas de Dados

- Escolha adequada impacta diretamente na performance
- STL oferece implementações otimizadas
- Customização necessária para casos específicos (BitBuffer)

Ferramentas

- C++ ideal para algoritmos de baixo nível
- GTK proporciona interface profissional
- Integração entre ferramentas é crucial